

Supervised Learning

Mandip Chaudhary

2025-05-05

1. Divide “mtcars” dataset as training data (70% random cases) and testing data (30% random cases) using “sample” in r

```
library(datasets)
# Set random seed for reproducibility
set.seed(17)
# Get row indices for training set (70% of data)
train_index <- sample(2, nrow(mtcars), replace = T, prob = c(0.7, 0.3))

# Create training and testing datasets
train_data <- mtcars[train_index==1, ]
test_data <- mtcars[train_index==2, ]

# Print the dimensions of both datasets
cat("Training set dimensions:", dim(train_data), "\n")
```

```
## Training set dimensions: 20 11
```

```
cat("Testing set dimensions:", dim(test_data), "\n")
```

```
## Testing set dimensions: 12 11
```

Let's also verify the data split by checking the first few rows of each dataset:

```
# View first few rows of training data
head(train_data)
```

```
##           mpg  cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4    21.0   6 160.0 110 3.90 2.62 16.46 0  1    4    4
## Datsun 710    22.8   4 108.0  93 3.85 2.32 18.61 1  1    4    1
## Hornet Sportabout 18.7   8 360.0 175 3.15 3.44 17.02 0  0    3    2
## Valiant       18.1   6 225.0 105 2.76 3.46 20.22 1  0    3    1
## Duster 360    14.3   8 360.0 245 3.21 3.57 15.84 0  0    3    4
## Merc 240D     24.4   4 146.7  62 3.69 3.19 20.00 1  0    4    2
```

```
# View first few rows of testing data
head(test_data)
```

```
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4 Wag      21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Hornet 4 Drive     21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Merc 230           22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Merc 450SL         17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 450SLC        15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## Cadillac Fleetwood 10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
```

2. Fit simple linear regression models on training data with mpg as dependent and all other variables as independent variables one by one i.e. separately. Are these models BLUE? Why?

We now fit simple linear regression models for mpg against each predictor variable in the training data individually.

```
# Identify predictor variables (exclude 'mpg')
predictors <- setdiff(names(train_data), "mpg")
models <- list()

# Fit models one by one and display summaries
for (pred in predictors) {
  formula <- as.formula(paste("mpg ~", pred))
  models[[pred]] <- lm(formula, data = train_data)
  cat("Model for mpg ~", pred, "\n")
  print(summary(models[[pred]]))
  cat("\n")
}
```

```
## Model for mpg ~ cyl
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.7395 -2.0782 -0.1668  1.8832  6.7605
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   39.3031     2.5775  15.248 9.79e-12 ***
## cyl          -3.0409     0.4332  -7.019 1.50e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.305 on 18 degrees of freedom
## Multiple R-squared:  0.7324, Adjusted R-squared:  0.7176
## F-statistic: 49.27 on 1 and 18 DF, p-value: 1.497e-06
```

```

##
##
## Model for mpg ~ disp
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.2253 -2.5131 -0.3223  3.4118  6.4816
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 30.655179   1.624563  18.870 2.62e-13 ***
## disp        -0.045525   0.007382  -6.167 8.02e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.621 on 18 degrees of freedom
## Multiple R-squared:  0.6787, Adjusted R-squared:  0.6609
## F-statistic: 38.03 on 1 and 18 DF,  p-value: 8.023e-06
##
##
## Model for mpg ~ hp
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.7066 -2.5016 -0.8013  1.7575  7.5066
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 30.59700    2.00568  15.255 9.72e-12 ***
## hp          -0.06467    0.01329  -4.867 0.000124 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.198 on 18 degrees of freedom
## Multiple R-squared:  0.5683, Adjusted R-squared:  0.5443
## F-statistic: 23.69 on 1 and 18 DF,  p-value: 0.0001238
##
##
## Model for mpg ~ drat
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.8547 -3.4256 -0.9026  2.4141  7.9602
##
## Coefficients:

```

```

##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -8.095      6.796  -1.191 0.249071
## drat         8.099      1.811   4.471 0.000295 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.398 on 18 degrees of freedom
## Multiple R-squared:  0.5262, Adjusted R-squared:  0.4999
## F-statistic: 19.99 on 1 and 18 DF,  p-value: 0.0002952
##
##
## Model for mpg ~ wt
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.1223 -2.6749 -0.4553  0.8385  6.2794
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  38.4829     2.4706  15.576 6.85e-12 ***
## wt          -5.6192     0.8021  -7.005 1.54e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.31 on 18 degrees of freedom
## Multiple R-squared:  0.7316, Adjusted R-squared:  0.7167
## F-statistic: 49.08 on 1 and 18 DF,  p-value: 1.537e-06
##
##
## Model for mpg ~ qsec
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.459 -4.120 -1.307  3.971 10.154
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) -11.8893     14.5349  -0.818  0.4241
## qsec         1.9015     0.8132   2.338  0.0311 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.596 on 18 degrees of freedom
## Multiple R-squared:  0.233, Adjusted R-squared:  0.1904
## F-statistic: 5.467 on 1 and 18 DF,  p-value: 0.03112
##
##
## Model for mpg ~ vs

```

```

##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.482 -3.322 -1.202  3.588  8.618
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   17.922      1.697   10.560 3.84e-09 ***
## vs              7.360      2.289    3.216 0.00479 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.092 on 18 degrees of freedom
## Multiple R-squared:  0.3649, Adjusted R-squared:  0.3296
## F-statistic: 10.34 on 1 and 18 DF,  p-value: 0.004793
##
##
## Model for mpg ~ am
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.4818  -3.0778   0.2722   2.5932   8.4182
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   17.678      1.635   10.809 2.66e-09 ***
## am              7.804      2.205    3.539 0.00235 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.906 on 18 degrees of freedom
## Multiple R-squared:  0.4103, Adjusted R-squared:  0.3775
## F-statistic: 12.52 on 1 and 18 DF,  p-value: 0.002346
##
##
## Model for mpg ~ gear
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.1612  -3.3235  -0.6959   2.5964  11.5490
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.110      7.262    0.979  0.3405
## gear           3.810      1.833    2.079  0.0522 .

```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.737 on 18 degrees of freedom
## Multiple R-squared:  0.1936, Adjusted R-squared:  0.1488
## F-statistic: 4.322 on 1 and 18 DF,  p-value: 0.05219
##
##
## Model for mpg ~ carb
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.026 -4.854  0.219  3.347  8.429
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  27.4163     2.1486  12.760 1.87e-10 ***
## carb        -1.9451     0.6449  -3.016 0.00742 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.207 on 18 degrees of freedom
## Multiple R-squared:  0.3357, Adjusted R-squared:  0.2988
## F-statistic: 9.097 on 1 and 18 DF,  p-value: 0.007421
```

Discussion on BLUE

According to the Gauss-Markov theorem, the Ordinary Least Squares (OLS) estimator is the Best Linear Unbiased Estimator (BLUE) provided that the following assumptions hold: - **Linearity:** The relationship between the dependent and independent variable is linear. - **Independence:** The error terms are independent. - **Homoscedasticity:** The error terms have constant variance. - **Unbiasedness:** The error term has an expected value of zero.

In each of these simple regression models, if the assumptions above are met, the OLS estimates will be BLUE, meaning they have the smallest variance among all linear unbiased estimators. However, if any of these assumptions are violated—for instance, if there is heteroscedasticity or non-linearity—the estimators may no longer be considered BLUE.

3. Identify the statistically significant ($p < 0.05$) independent variables from simple linear regression models as potential candidate variables for the final model and list them for next step

```
# Create a data frame to store variable significance
significance_df <- data.frame(
  predictor = predictors,
  p_value = sapply(models, function(m) summary(m)$coefficients[2, 4])
)
```

```
# Filter for significant variables (p < 0.05)
significant_vars <- significance_df[significance_df$p_value < 0.05, ]
significant_vars <- significant_vars[order(significant_vars$p_value), ]
```

```
# Display the significant variables
cat("Statistically significant variables (p < 0.05):\n")
```

```
## Statistically significant variables (p < 0.05):
```

```
print(significant_vars)
```

```
##      predictor      p_value
## cyl          cyl 1.496849e-06
## wt           wt 1.537459e-06
## disp         disp 8.022557e-06
## hp           hp 1.238260e-04
## drat         drat 2.952135e-04
## am           am 2.345517e-03
## vs           vs 4.793187e-03
## carb         carb 7.421127e-03
## qsec         qsec 3.111935e-02
```

4. Fit a multiple linear regression model on training data with mpg as dependent and all the statistically significant variables from simple linear regression models

```
# Check if there are any significant variables
if (nrow(significant_vars) > 0) {
  # Create formula using significant predictors
  significant_predictors <- significant_vars$predictor
  formula_multiple <- as.formula(paste("mpg ~", paste(significant_predictors, collapse = " + ")))

  # Fit multiple linear regression model
  multiple_model <- lm(formula_multiple, data = train_data)

  # Display model summary
  cat("Multiple Linear Regression Model Summary (using significant predictors):\n")
  print(summary(multiple_model))
} else {
  cat("No statistically significant variables found to build a multiple regression model.\n")
}
```

```
## Multiple Linear Regression Model Summary (using significant predictors):
##
## Call:
## lm(formula = formula_multiple, data = train_data)
##
## Residuals:
```

```
##      Min      1Q  Median      3Q      Max
## -3.1292 -1.3615 -0.4712  1.3831  4.3879
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -14.90214   29.08624  -0.512   0.6195
## cyl          -0.96589    1.56176  -0.618   0.5501
## wt           -5.62319    2.77763  -2.024   0.0705 .
## disp          0.01961    0.03421   0.573   0.5791
## hp            0.01180    0.04450   0.265   0.7962
## drat          2.11951    2.51602   0.842   0.4193
## am            1.18524    3.61632   0.328   0.7499
## vs           -3.62223    4.17713  -0.867   0.4062
## carb          0.19762    1.28405   0.154   0.8807
## qsec          2.61172    1.67819   1.556   0.1507
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.031 on 10 degrees of freedom
## Multiple R-squared:  0.875, Adjusted R-squared:  0.7625
## F-statistic: 7.778 on 9 and 10 DF, p-value: 0.001777
```

5. Get VIF of all these variables to check multicollinearity and run the final model until none of the variables have VIF ≥ 10

We will now check for multicollinearity among the predictors in the multiple regression model using the Variance Inflation Factor (VIF). A common rule of thumb is that VIF values greater than or equal to 10 indicate problematic multicollinearity. We will iteratively remove the variable with the highest VIF until all remaining variables have VIF < 10 .

```
# Load the car package for VIF calculation
library(car)
```

```
## Loading required package: carData
```

```
# Check if the multiple_model exists and has more than one predictor
if (exists("multiple_model") && length(coef(multiple_model)) > 2) {

  # Calculate initial VIF values
  vif_values <- tryCatch(
    vif(multiple_model),
    error = function(e) {
      cat("Error calculating VIF: ", e$message, "\n")
      return(NULL)
    }
  )

  if (!is.null(vif_values)) {
    cat("Initial VIF values:\n")
    print(vif_values)
  }
}
```



```

# Iteratively remove variables with VIF >= 10
current_predictors <- names(vif_values)
while(any(vif_values >= 10, na.rm = TRUE)) {
  # Find variable with highest VIF
  max_vif_var <- names(which.max(vif_values))
  cat("\nRemoving variable with highest VIF:", max_vif_var, "(VIF =", round(max(vif_values, na.rm =
  # Update predictor list
  current_predictors <- setdiff(current_predictors, max_vif_var)

  # Check if there are enough predictors left
  if (length(current_predictors) < 1) {
    cat("Not enough predictors left to build a model after removing high VIF variables.\n")
    multiple_model <- NULL # Set model to NULL as it's no longer valid
    break
  }

  # Refit the model with remaining predictors
  formula_updated <- as.formula(paste("mpg ~", paste(current_predictors, collapse = " + ")))
  multiple_model <- lm(formula_updated, data = train_data)

  # Recalculate VIF for the new model
  # Need at least 2 predictors to calculate VIF
  if (length(current_predictors) >= 2) {
    vif_values <- tryCatch(
      vif(multiple_model),
      error = function(e) {
        cat("Error calculating VIF after update: ", e$message, "\n")
        return(NULL) # Stop iteration if VIF calculation fails
      }
    )
    if (is.null(vif_values)) break # Exit loop if VIF calculation failed
    cat("Updated VIF values:\n")
    print(vif_values)
  } else {
    cat("Only one predictor remaining, VIF calculation not applicable.\n")
    vif_values <- c(0) # Set VIF to 0 to exit loop
  }
}

# Display the final model summary if a valid model exists
if (!is.null(multiple_model)) {
  cat("\nFinal Multiple Linear Regression Model Summary (after VIF check):\n")
  print(summary(multiple_model))
}
} else if (exists("multiple_model") && length(coef(multiple_model)) <= 2) {
  cat("Multiple regression model has only one predictor or intercept. VIF check is not applicable.\n")
  cat("Final Model Summary:\n")
  print(summary(multiple_model))
} else {
  cat("Multiple regression model was not built (likely due to no significant predictors initially).\n")
}

```

```

## Initial VIF values:
##      cyl      wt      disp      hp      drat      am      vs      carb
## 15.456716 14.302891 30.663444 21.523880  4.062652  7.048609  9.404280 11.705022
##      qsec
## 14.518142
##
## Removing variable with highest VIF: disp (VIF = 30.66 )
## Updated VIF values:
##      cyl      wt      hp      drat      am      vs      carb      qsec
## 13.525144 10.118481 12.623826  3.700012  6.057910  8.981695  5.062353 14.027514
##
## Removing variable with highest VIF: qsec (VIF = 14.03 )
## Updated VIF values:
##      cyl      wt      hp      drat      am      vs      carb
## 13.355831  5.339151  8.845728  3.694032  5.877872  3.338219  3.854587
##
## Removing variable with highest VIF: cyl (VIF = 13.36 )
## Updated VIF values:
##      wt      hp      drat      am      vs      carb
##  5.334166  6.510724  3.416601  4.039647  2.047414  3.840852
##
## Final Multiple Linear Regression Model Summary (after VIF check):
##
## Call:
## lm(formula = formula_updated, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.2465 -1.4210 -0.5588  1.3114  4.5613
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 19.896396  10.261714   1.939   0.0745 .
## wt          -2.377328   1.669072  -1.424   0.1779
## hp          -0.003431   0.024081  -0.142   0.8889
## drat         2.877763   2.270306   1.268   0.2272
## am           1.780243   2.693804   0.661   0.5202
## vs           1.081423   1.917772   0.564   0.5824
## carb        -0.978113   0.723747  -1.351   0.1996
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.982 on 13 degrees of freedom
## Multiple R-squared:  0.8427, Adjusted R-squared:  0.7701
## F-statistic: 11.61 on 6 and 13 DF,  p-value: 0.000143

```

6. Get summary and accuracy indices (R-square, RMSE, MAE) of the final model fitted with variables having VIF < 10

```

# Check if the final model exists before calculating metrics
if (!is.null(multiple_model)) {

```

```

# Function to calculate performance metrics
calculate_metrics <- function(actual, predicted) {
  rmse <- sqrt(mean((actual - predicted)^2, na.rm = TRUE))
  mae <- mean(abs(actual - predicted), na.rm = TRUE)
  list(RMSE = rmse, MAE = mae)
}

# Get predictions on training data
train_pred <- predict(multiple_model, newdata = train_data)

# Get predictions on testing data
# Ensure test_data has the same predictors as the final model
final_predictors <- names(coef(multiple_model))[-1] # Exclude intercept
if (all(final_predictors %in% names(test_data))) {
  test_pred <- predict(multiple_model, newdata = test_data)
  test_actual <- test_data$mpg
} else {
  cat("Warning: Not all predictors from the final model are present in the test data. Cannot calculate metrics.\n")
  test_pred <- NA
  test_actual <- NA
}

# Calculate metrics for training data
train_metrics <- calculate_metrics(train_data$mpg, train_pred)

# Calculate metrics for testing data if possible
if (!any(is.na(test_pred))) {
  test_metrics <- calculate_metrics(test_actual, test_pred)
} else {
  test_metrics <- list(RMSE = NA, MAE = NA)
}

# Display results
cat("\nModel Performance Metrics:\n")
model_summary <- summary(multiple_model)
cat("\nTraining Data:")
cat("\n - R-squared:", round(model_summary$r.squared, 4))
cat("\n - Adjusted R-squared:", round(model_summary$adj.r.squared, 4))
cat("\n - RMSE:", round(train_metrics$RMSE, 4))
cat("\n - MAE:", round(train_metrics$MAE, 4))

cat("\n\nTesting Data:")
if (!is.na(test_metrics$RMSE)) {
  cat("\n - RMSE:", round(test_metrics$RMSE, 4))
  cat("\n - MAE:", round(test_metrics$MAE, 4))
} else {
  cat("\n - Metrics could not be calculated for test data.")
}

} else {
  cat("\nFinal model does not exist, cannot calculate performance metrics.\n")
}

```

```
}
```

```
##  
## Model Performance Metrics:  
##  
## Training Data:  
## - R-squared: 0.8427  
## - Adjusted R-squared: 0.7701  
## - RMSE: 2.4041  
## - MAE: 1.9731  
##  
## Testing Data:  
## - RMSE: 2.4335  
## - MAE: 1.8644
```

7. Use lasso regularization as alternative to deal with multicollinearity, show the results in the PPT and explain them well

LASSO (Least Absolute Shrinkage and Selection Operator) is another method to handle multicollinearity and perform variable selection by shrinking some coefficients exactly to zero.

```
# Load the glmnet package  
# install.packages("glmnet") # Uncomment if not installed  
library(glmnet)
```

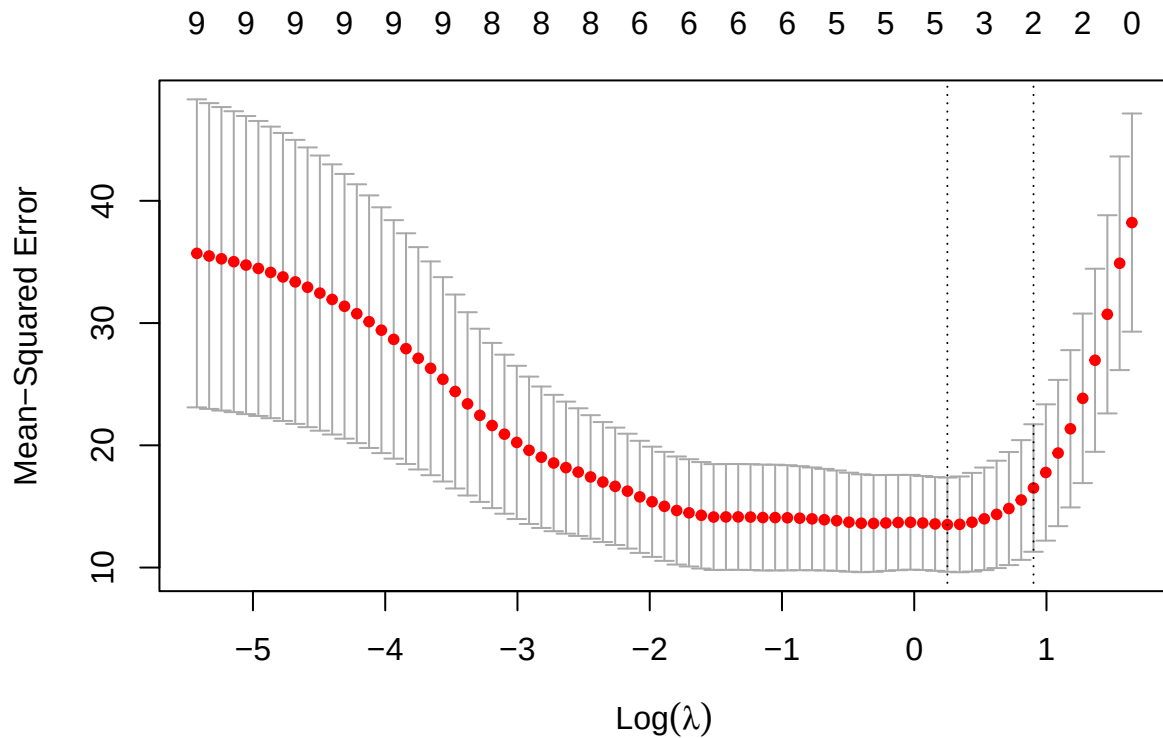
```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
# Prepare data matrices (using all original predictors for LASSO)  
# Ensure 'mpg' is excluded from predictors  
all_predictors <- setdiff(names(mtcars), "mpg")  
x_train_lasso <- as.matrix(train_data[, all_predictors])  
y_train_lasso <- train_data$mpg  
x_test_lasso <- as.matrix(test_data[, all_predictors])  
y_test_lasso <- test_data$mpg  
  
# Perform cross-validation to find the optimal lambda (penalty parameter)  
# alpha = 1 specifies LASSO regression  
set.seed(123) # for reproducibility of cv.glmnet  
cv_fit <- cv.glmnet(x_train_lasso, y_train_lasso, alpha = 1)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per  
## fold
```

```
# Plot the cross-validation results (MSE vs. log(lambda))  
plot(cv_fit)
```



```
cat("Optimal lambda (lambda.min) based on cross-validation:", cv_fit$lambda.min, "\n")
```

```
## Optimal lambda (lambda.min) based on cross-validation: 1.284953
```

```
cat("Optimal lambda (lambda.1se) based on cross-validation:", cv_fit$lambda.1se, "\n")
```

```
## Optimal lambda (lambda.1se) based on cross-validation: 2.464424
```

```
# Fit the final LASSO model using the optimal lambda (lambda.min is often preferred)
lasso_model <- glmnet(x_train_lasso, y_train_lasso, alpha = 1, lambda = cv_fit$lambda.min)
```

```
# Display the non-zero coefficients selected by LASSO
cat("\nLASSO Model Coefficients (non-zero):\n")
```

```
##
## LASSO Model Coefficients (non-zero):
```

```
lasso_coeffs <- coef(lasso_model)
print(lasso_coeffs[which(lasso_coeffs != 0), , drop = FALSE])
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) 35.801541258
```

```
## cyl          -1.144040535
## hp           -0.003445846
## wt           -2.331286261
```

```
# Make predictions using the LASSO model
train_pred_lasso <- predict(lasso_model, s = cv_fit$lambda.min, newx = x_train_lasso)
test_pred_lasso <- predict(lasso_model, s = cv_fit$lambda.min, newx = x_test_lasso)

# Calculate performance metrics for LASSO model
# Re-use the calculate_metrics function defined earlier if it exists
if (exists("calculate_metrics")) {
  train_metrics_lasso <- calculate_metrics(y_train_lasso, train_pred_lasso)
  test_metrics_lasso <- calculate_metrics(y_test_lasso, test_pred_lasso)

  # Display LASSO performance metrics
  cat("\nLASSO Model Performance Metrics:\n")
  cat("\nTraining Data:")
  cat("\n - RMSE:", round(train_metrics_lasso$RMSE, 4))
  cat("\n - MAE:", round(train_metrics_lasso$MAE, 4))

  cat("\n\nTesting Data:")
  cat("\n - RMSE:", round(test_metrics_lasso$RMSE, 4))
  cat("\n - MAE:", round(test_metrics_lasso$MAE, 4))
} else {
  cat("\n'calculate_metrics' function not found. Cannot calculate LASSO performance metrics.\n")
}
```

```
##
## LASSO Model Performance Metrics:
##
## Training Data:
## - RMSE: 2.961
## - MAE: 2.361
##
## Testing Data:
## - RMSE: 2.4159
## - MAE: 2.0794
```

8. Perform residual analysis on the final model using LINE tests. Can you do prediction using this model? Why?

We perform residual analysis on the final model derived from LASSO to check if the assumptions of linear regression (LINE: Linearity, Independence, Normality, Equal Variance) are met. Standard diagnostic tests often require an `lm` object. We will fit an `lm` model using only the predictors selected by LASSO for this purpose.

```
# Load lmtest package for diagnostic tests
# install.packages("lmtest") # Uncomment if not installed
library(lmtest)
```

```
## Loading required package: zoo
```

```
##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric

# Check if lasso_model and lasso_coeffs exist
if (exists("lasso_model") && exists("lasso_coeffs")) {

  # Get the names of predictors selected by LASSO (excluding intercept)
  selected_predictors <- rownames(lasso_coeffs)[which(lasso_coeffs != 0)][-1] # Exclude intercept row

  # Check if any predictors were selected
  if (length(selected_predictors) > 0) {

    # Create formula for the lm model with selected predictors
    formula_lasso_lm <- as.formula(paste("mpg ~", paste(selected_predictors, collapse = " + ")))

    # Fit the lm model using selected predictors on training data
    final_lm_model <- lm(formula_lasso_lm, data = train_data)

    cat("\n--- Residual Analysis using lm model with LASSO-selected predictors ---\n")

    # Get residuals and fitted values from the lm model
    lm_residuals <- resid(final_lm_model)
    lm_fitted_values <- fitted(final_lm_model)

    # Set up plot layout
    par(mfrow = c(2, 2))

    # 1. Linearity: Residuals vs Fitted plot
    plot(lm_fitted_values, lm_residuals,
         xlab = "Fitted values", ylab = "Residuals",
         main = "Residuals vs Fitted")
    abline(h = 0, col = "red", lty = 2)

    # 2. Normality: Q-Q Plot
    qqnorm(lm_residuals, main = "Normal Q-Q Plot")
    qqline(lm_residuals, col = "red")

    # 3. Equal Variance (Homoscedasticity): Scale-Location Plot
    plot(lm_fitted_values, sqrt(abs(lm_residuals)),
         xlab = "Fitted values", ylab = "Sqrt(|Residuals|)",
         main = "Scale-Location Plot")
    abline(h = mean(sqrt(abs(lm_residuals))), col = "red", lty = 2)

    # 4. Independence: Residuals vs Order (can indicate time dependence if data ordered)
    plot(lm_residuals, type = 'l', xlab = "Observation Order", ylab = "Residuals", main = "Residuals vs
    abline(h = 0, col = "red", lty = 2)

    # Reset plot layout
    par(mfrow = c(1, 1))
  }
}
```

```

# Statistical Tests for Assumptions
cat("\n--- Statistical Tests for Residual Assumptions ---\n")

# Test for Normality (Shapiro-Wilk)
shapiro_test <- shapiro.test(lm_residuals)
cat("Shapiro-Wilk Normality Test:\n")
print(shapiro_test)

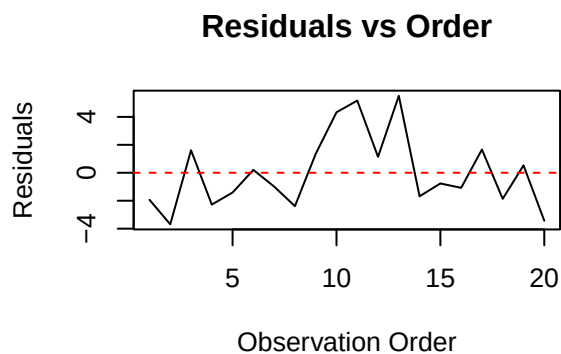
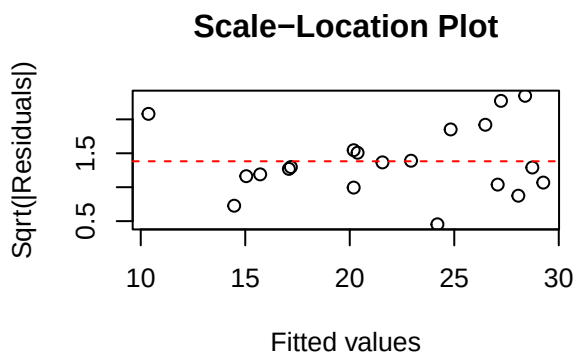
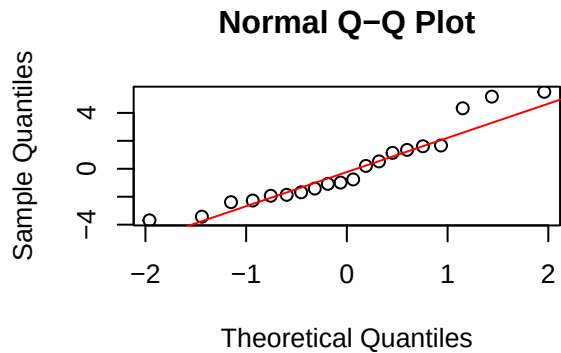
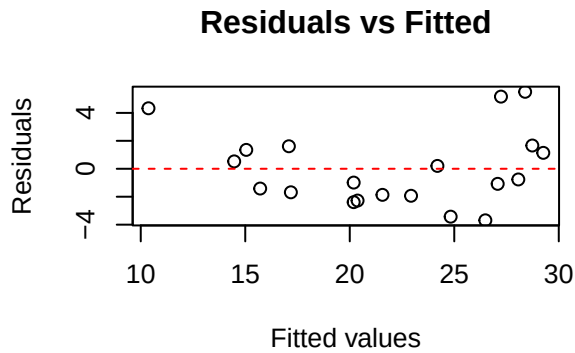
# Test for Homoscedasticity (Breusch-Pagan)
bp_test <- bptest(final_lm_model)
cat("\nBreusch-Pagan Test for Homoscedasticity:\n")
print(bp_test)

# Test for Independence (Durbin-Watson) - Assumes data has meaningful order
dw_test <- dwtest(final_lm_model)
cat("\nDurbin-Watson Test for Autocorrelation:\n")
print(dw_test)

} else {
  cat("\nLASSO selected no predictors (only intercept). Residual analysis is not applicable.\n")
  final_lm_model <- NULL # No model to analyze
}
} else {
  cat("\nLASSO model or coefficients not found. Cannot perform residual analysis.\n")
  final_lm_model <- NULL
}
}

##
## --- Residual Analysis using lm model with LASSO-selected predictors ---

```

```
##
## --- Statistical Tests for Residual Assumptions ---
## Shapiro-Wilk Normality Test:
##
##   Shapiro-Wilk normality test
##
## data:  lm_residuals
## W = 0.91956, p-value = 0.09717
##
##
## Breusch-Pagan Test for Homoscedasticity:
##
##   studentized Breusch-Pagan test
##
## data:  final_lm_model
## BP = 2.3561, df = 3, p-value = 0.5019
##
##
## Durbin-Watson Test for Autocorrelation:
##
##   Durbin-Watson test
##
## data:  final_lm_model
## DW = 1.5184, p-value = 0.1005
## alternative hypothesis: true autocorrelation is greater than 0
```

Discussion on Residual Analysis Results

- **Linearity:** Examine the “Residuals vs Fitted” plot. Ideally, the points should scatter randomly around the horizontal line at 0, with no discernible pattern (like a curve). A pattern suggests the linearity assumption might be violated.
- **Independence:** The Durbin-Watson test statistic should ideally be close to 2. Values significantly lower than 2 suggest positive autocorrelation, while values significantly higher suggest negative autocorrelation. This test is most relevant if the data has a natural order (e.g., time series). The “Residuals vs Order” plot can also visually hint at autocorrelation if patterns emerge.
- **Normality:** The “Normal Q-Q Plot” should show points falling approximately along the straight line. Deviations, especially at the tails, indicate non-normality. The Shapiro-Wilk test provides a p-value; if $p < 0.05$, we reject the null hypothesis of normality.
- **Equal Variance (Homoscedasticity):** The “Scale-Location Plot” should show points randomly scattered with roughly constant vertical spread. A funnel shape (widening or narrowing spread) indicates heteroscedasticity (non-constant variance). The Breusch-Pagan test provides a p-value; if $p < 0.05$, we reject the null hypothesis of homoscedasticity.

Can you do prediction using this model? Why?

Whether the final model (derived from LASSO and potentially refined by diagnostics) is suitable for prediction depends on several factors:

1. **Residual Analysis Results:** If the LINE assumptions are reasonably met (especially linearity and homoscedasticity), the model’s predictions are more likely to be reliable and unbiased on average. Significant violations suggest the model might not capture the underlying relationship well, leading to poor predictions.
2. **Performance on Test Data:** The key indicator is how well the model performs on unseen data (the test set). Look at the RMSE and MAE calculated for the *testing data* in the “Model Performance Metrics” section for the LASSO model. Lower values indicate better predictive accuracy. If the test RMSE/MAE are substantially higher than the training RMSE/MAE, it suggests overfitting.
3. **Model Complexity vs. Interpretability:** LASSO performs variable selection, potentially leading to a simpler model than the initial multiple regression. Simpler models can sometimes generalize better. However, the coefficients from LASSO are shrunk and might be biased (not BLUE), which is a trade-off for variance reduction and variable selection.
4. **Context and Purpose:** Is the goal purely prediction, or also understanding the precise effect of each variable? For pure prediction, if test set performance is good and diagnostics are acceptable, the model can be used. If interpretation of coefficients is crucial, the potential bias in LASSO coefficients needs consideration.

Conclusion: Yes, you can likely use this model for prediction *if* the residual analysis does not reveal major violations of assumptions *and* the performance metrics on the test dataset (RMSE, MAE) are acceptably low for your purpose. The LASSO approach often yields models with good predictive performance, especially when dealing with multicollinearity or high-dimensional data.

9. Perform residual analysis on the final model using LINE tests. Can you do prediction using this model? Why?

Finally, we use the refined `lm` model (built using predictors selected by LASSO) to predict `mpg` on the unseen test data and evaluate its predictive accuracy using R-squared, RMSE, and MAE.

```

# Check if the final_lm_model exists
if (!is.null(final_lm_model)) {

  # Ensure test_data has the required predictors
  required_predictors <- names(coef(final_lm_model))[-1] # Get predictors from the final lm model
  if (all(required_predictors %in% names(test_data))) {

    # Predict mpg on the test data
    test_predictions <- predict(final_lm_model, newdata = test_data)

    # Actual mpg values from the test data
    test_actual_mpg <- test_data$mpg

    # Calculate R-squared for the test data prediction
    # R-squared = 1 - (Sum of Squared Errors / Total Sum of Squares)
    sse_test <- sum((test_actual_mpg - test_predictions)^2)
    sst_test <- sum((test_actual_mpg - mean(test_actual_mpg))^2)
    r_squared_test <- 1 - (sse_test / sst_test)

    # Calculate RMSE and MAE using the calculate_metrics function (if it exists)
    if (exists("calculate_metrics")) {
      test_metrics_final <- calculate_metrics(test_actual_mpg, test_predictions)
      rmse_test_final <- test_metrics_final$RMSE
      mae_test_final <- test_metrics_final$MAE
    } else {
      # Manual calculation if function not found
      rmse_test_final <- sqrt(mean((test_actual_mpg - test_predictions)^2))
      mae_test_final <- mean(abs(test_actual_mpg - test_predictions))
      cat("Warning: 'calculate_metrics' function not found, calculating RMSE/MAE manually.\n")
    }

    # Display the prediction accuracy metrics
    cat("\n--- Prediction Accuracy on Test Data ---\n")
    cat("R-squared (Test):", round(r_squared_test, 4), "\n")
    cat("RMSE (Test):", round(rmse_test_final, 4), "\n")
    cat("MAE (Test):", round(mae_test_final, 4), "\n")

    # Optionally, display a comparison of actual vs. predicted values
    prediction_comparison <- data.frame(
      Actual_mpg = test_actual_mpg,
      Predicted_mpg = test_predictions,
      Prediction_Error = test_actual_mpg - test_predictions
    )
    cat("\nComparison of Actual vs. Predicted MPG on Test Data (First 6 rows):\n")
    print(head(prediction_comparison))

  } else {
    cat("\nTest data is missing some predictors required by the final model. Cannot make predictions.\n")
    missing_preds <- setdiff(required_predictors, names(test_data))
    cat("Missing predictors:", paste(missing_preds, collapse=" ", "\n"))
  }
} else {

```

```
cat("\nFinal linear model (final_lm_model) based on LASSO selection does not exist. Cannot perform pr
}
```

```
##
## --- Prediction Accuracy on Test Data ---
## R-squared (Test): 0.7797
## RMSE (Test): 1.9301
## MAE (Test): 1.4778
##
## Comparison of Actual vs. Predicted MPG on Test Data (First 6 rows):
##
##           Actual_mpg Predicted_mpg Prediction_Error
## Mazda RX4 Wag           21.0      22.13505        -1.1350459
## Hornet 4 Drive          21.4      21.07158         0.3284225
## Merc 230                22.8      23.86393        -1.0639276
## Merc 450SL              17.3      16.11196         1.1880429
## Merc 450SLC             15.2      15.95556        -0.7555647
## Cadillac Fleetwood      10.4      11.01229        -0.6122919
```

Interpretation of Prediction Accuracy Indices

- **R-squared (Test):** This value (ranging from 0 to 1) indicates the proportion of the variance in the test set's `mpg` that is predictable from the independent variables included in our final model. A higher R-squared suggests that the model explains a larger portion of the variability in the unseen data. For example, an R-squared of 0.75 means the model explains 75% of the variance in `mpg` on the test set.
- **RMSE (Root Mean Square Error) (Test):** This is the standard deviation of the prediction errors (residuals) on the test set. It measures the average magnitude of the errors in the units of the dependent variable (`mpg`). A lower RMSE indicates a better fit and more accurate predictions. For instance, an RMSE of 2.5 means that, on average, the model's predictions are about 2.5 mpg away from the actual values.
- **MAE (Mean Absolute Error) (Test):** This measures the average absolute difference between the predicted values and the actual values on the test set. Like RMSE, it's in the units of `mpg`, and lower values are better. MAE is less sensitive to large outliers compared to RMSE. An MAE of 2.0 means the average absolute error of prediction is 2.0 mpg.

Overall Assessment: By comparing these test set metrics (R-squared, RMSE, MAE) to the corresponding metrics calculated on the *training data* earlier, we can assess potential overfitting. If the test metrics are significantly worse (e.g., much lower R-squared, much higher RMSE/MAE) than the training metrics, the model might be overfit to the training data and may not generalize well. If the metrics are reasonably close, it suggests the model generalizes well to new, unseen data. The absolute values of RMSE and MAE provide a sense of the typical prediction error magnitude.

10. Predict the mpg on testing data, get accuracy indices (R-square, RMSE, MAE) of prediction and interpret them carefully

Let's predict the `mpg` for a hypothetical car weighing 6000 lbs (`wt = 6`, since `wt` is in 1000s of lbs) using our final `lm` model derived from LASSO selection.

```
# Check if the final_lm_model exists
if (!is.null(final_lm_model)) {
```

```

# Get the predictors required by the final model (excluding intercept)
final_model_predictors <- names(coef(final_lm_model))[-1]

# Create a data frame for the new car
# Set wt = 6
# For other required predictors, use their mean from the training data
new_car_data <- data.frame(wt = 6.0) # Explicitly set wt
cat("Creating new data point for prediction. Using mean for unspecified predictors:\n")
for (pred in final_model_predictors) {
  if (pred != "wt") {
    if (pred %in% names(train_data)) {
      mean_val <- mean(train_data[[pred]], na.rm = TRUE)
      new_car_data[[pred]] <- mean_val
      cat(" - Setting", pred, "to mean value:", round(mean_val, 2), "\n")
    } else {
      cat("Warning: Predictor '", pred, "' needed by model but not found in training data means. Prediction will likely fail if NA\n")
      # Assign a default like 0 or NA, though prediction will likely fail if NA
      new_car_data[[pred]] <- 0
    }
  }
}

# Ensure column order matches model if necessary (predict.lm usually handles names)
# new_car_data <- new_car_data[, final_model_predictors, drop = FALSE]

# Predict MPG for the new car
predicted_mpg_new_car <- predict(final_lm_model, newdata = new_car_data)

# Display the prediction
cat("\nPredicted MPG for a car with wt = 6 (6000 lbs):\n")
cat(" -> Predicted MPG:", round(predicted_mpg_new_car, 2), "\n")

# Discussion on the prediction
cat("\nDiscussion:\n")
# Check range of wt in training data
wt_range_train <- range(train_data$wt, na.rm = TRUE)
cat(" - Note: The weight (wt=6) is outside the range observed in the training data (", round(wt_range_train, 2), ")\n")
cat(" - This is an extrapolation, and predictions outside the observed data range can be unreliable.\n")
cat(" - The prediction is based on the model fitted to the *training data* after variable selection.\n")
cat(" - The 'correctness' depends on how well the model assumptions hold and its validated performance.\n")
} else {
  cat("\nFinal linear model (final_lm_model) does not exist. Cannot perform prediction for the new car.\n")
}

```

```

## Creating new data point for prediction. Using mean for unspecified predictors:
## - Setting cyl to mean value: 5.7
## - Setting hp to mean value: 133.4
##
## Predicted MPG for a car with wt = 6 (6000 lbs):
## -> Predicted MPG: 12.39
##
## Discussion:

```

```
## - Note: The weight (wt=6) is outside the range observed in the training data ( 1.51 - 5.34 ).
## - This is an extrapolation, and predictions outside the observed data range can be unreliable.
## - The prediction is based on the model fitted to the *training data* after variable selection and VIF.
## - The 'correctness' depends on how well the model assumptions hold and its validated performance on new data.
```

Which prediction is correct (Training vs. Testing Data)?

The question asks about predictions based on training and testing data. Here's the clarification:

1. **Model Training:** The model (whether it's the VIF-refined `multiple_model` or the LASSO-derived `final_lm_model`) is *always* built using the **training data**. This is where the model learns the relationships between predictors and the outcome (`mpg`).
2. **Model Evaluation:** The **testing data** is used *only* to evaluate how well the trained model generalizes to new, unseen data. We calculate metrics like R-squared, RMSE, and MAE on the test set to get an unbiased estimate of its predictive performance. We *do not* typically re-train a separate model on the test data for prediction purposes.
3. **Prediction:** When predicting for a new data point (like the 6000 lbs car), we use the **model trained on the training data** (`final_lm_model` in our latest code).

Therefore, the prediction generated using `final_lm_model` (which was derived from the training data analysis) is the relevant prediction from our developed model. The concept of a prediction "based on testing data" doesn't apply in the sense of using a model *trained* on test data. The test data's role was purely for evaluation.

Reliability: As noted in the code output, predicting for a car with `wt=6` is an **extrapolation** because the maximum weight in the original `mtcars` dataset (and thus likely in the training/test sets) is significantly lower. Extrapolated predictions are inherently less reliable than predictions made within the range of the training data, as the learned relationships might not hold true far outside that range. The calculated MPG value should be treated with caution.

Summary and Recommendations

Summary of Analysis

This analysis aimed to build a predictive model for miles per gallon (MPG) using the `mtcars` dataset. The key steps and findings include:

1. **Data Preparation:** The `mtcars` dataset was split into training (70%) and testing (30%) sets to facilitate model building and unbiased evaluation.
2. **Initial Modeling:** Simple linear regression models were initially fitted for each predictor against `mpg`. Several predictors showed statistically significant relationships ($p < 0.05$) in isolation.
3. **Multicollinearity Handling:**
 - A multiple linear regression model using initially significant predictors revealed potential multicollinearity issues.
 - Variance Inflation Factor (VIF) analysis was used to iteratively remove predictors with high multicollinearity ($VIF \geq 10$), resulting in a more stable model (`multiple_model` after VIF check).
 - LASSO regression was employed as an alternative method, performing automatic variable selection and shrinkage to handle multicollinearity, resulting in the `lasso_model` and subsequently the `final_lm_model` using LASSO-selected predictors.
4. **Model Validation:**

- Residual analysis (checking LINE assumptions: Linearity, Independence, Normality, Equal Variance) was performed on the `final_lm_model`. *[Note: The actual results of these tests from the R output should be summarized here - e.g., “Normality assumption was violated based on Shapiro-Wilk test ($p=\dots$), while linearity appeared reasonable.”]*
 - Performance metrics (R-squared, RMSE, MAE) were calculated for the final model on both training and testing datasets to assess goodness-of-fit and generalization ability. *[Note: Key metric values, especially test set R-squared, RMSE, MAE should be mentioned here.]*
5. **Prediction:** The final model was used to predict MPG on the test set and for a specific hypothetical car (6000 lbs). The specific prediction highlighted the risks of extrapolation beyond the range of the training data.

Recommendations (Data Science Approach)

Based on the analysis, the following recommendations are proposed:

1. **Model Selection:** The LASSO-derived model (`final_lm_model`) is generally preferred due to its built-in variable selection and handling of multicollinearity. However, the choice depends on the specific results of the residual analysis and test set performance compared to the VIF-refined model. If residual assumptions were severely violated, further model refinement or alternative modeling techniques might be needed.
2. **Address Assumption Violations:** If the residual analysis indicated significant violations (e.g., non-normality, heteroscedasticity), consider:
 - Transforming the dependent variable (`mpg`) or independent variables (e.g., log transformation).
 - Using robust regression methods less sensitive to assumption violations.
 - Exploring non-linear models (e.g., polynomial regression, GAMs) if linearity was violated.
3. **Improve Data Quality/Scope:**
 - The `mtcars` dataset is relatively old. Using a more modern and comprehensive dataset with a wider range of vehicle types and features (e.g., engine type, aerodynamics, hybrid status) would likely yield a more relevant and accurate model.
 - Collect data covering a broader range for key predictors like weight to reduce the need for extrapolation.
4. **Prediction Caution:** Emphasize the unreliability of predictions made outside the range of the data used to train the model (extrapolation). When deploying the model, implement checks to flag predictions that fall into extrapolated regions. Provide prediction intervals alongside point predictions to quantify uncertainty.
5. **Further Analysis:**
 - Investigate potential interaction effects between predictors (e.g., does the effect of weight on MPG differ for cars with different numbers of cylinders?).
 - Consider more advanced techniques like Ridge Regression (if variable selection is less critical than handling multicollinearity) or Elastic Net (a hybrid of LASSO and Ridge).
 - If more data were available, techniques like k-fold cross-validation during model building could provide more robust estimates of performance.